Third Research/Programming Assignment

Juliana Ng

MSDS 458 SEC 56

February 23, 2025

Abstract

This paper explores the performance of Recurrent Neural Networks (RNNs), Long-Short Term Memory networks (LSTMs), and Convolutional Neural Networks (CNNs) at text classification for news articles. Different model architectures are implemented to understand their effects on model performance. This research uses the AG news dataset containing 120,000 and 7,600 news articles across 4 categories for training and testing. TensorFlow and Keras are used to build the models and NTLK is used to pre-process the text data. The best performing model of this research uses 96 tokens for the top 1000 most frequent tokens in the corpus. It has an embedding size of 256, bidirectional and dense layers with 128 nodes and 2 dropout layers (0.3, 0.5). It ran for 5m and had a testing accuracy of 88% and loss of 0.33. The loss curves show an appropriate fitting model and confusion matrix shows a great classification accuracy of articles to their labeled classes.

Introduction

Recurrent Neural Network (RNN) is a neural network that processes sequential data like time-series data and text data, where the position of the data is relevant to the semantics. It has a short-term memory of recent data but faces challenges retaining past information and is susceptible to the exploding/vanishing gradients. (Built In, 2021) Long-Short Term Memory network (LSTM) is a type of RNN. It has the ability to retain information for a longer term because its memory cell is controlled by the input, forget, and output gates. (GeeksforGeeks)

The AG news consists of more than a million news articles collected from more than 2,000 news outlets. The dataset contains 120,000 articles for training and 7,600 articles for testing across 4 categories: World, Sports, Business, Sci/Tech. (Gulli & Ferro, 2006; Papers with Code)

In this paper, the AG news dataset is used to train Recurrent Neural Networks (RNNs), Long-Short Term Memory networks (LSTM), and Convolutional Neural Networks (CNNs) to find the best model by conducting a series of experiments adjusting the vocabulary size, model architecture, and regularization techniques. Regularization techniques are employed to minimize overfitting of models. We would like to train a model with high accuracy, minimal loss across training, validation, and testing stages and correctly classify the news articles with the labeled category.

Literature review

Luan and Lin used a variation of CNN and LSTM models to evaluate if the text dataset is subjective or objective. (Luan & Lin, 2019) The dataset was taken from Pang & Lee's paper where 5,000 movie reviews collected from www.rottentomatoes.com and 5,000 sentences from movies plot summaries from IMDB.com. They used 256 embedding dimensions and varied the size of the filter 3-5 where the number of filters is 128. They experimented with 6 models with different combinations of CNN, LSTM, COIF-LSTM (a variation of LSTM) and models without activation function and all models had precision, recall, and F1 score above 98.5%.

Methods

This research is conducted using the AG news dataset, which is split into 114,000 articles for training, 6,000 for validation, and 7,600 for testing. The main libraries are Tensorflow (v 2.18.0), Keras (v 3.8.0), NLTK, matplotlib, and scikit-learn libraries. The experiment is run on Google CoLaboratory using T4 GPU.

In each experiment, one hyperparameter is tweaked to find a model with the best accuracy, loss score, RMSE, and correct classification of the articles with the labelled class. All models are run for 10 epochs in a batch size of 32. Rmsprop is used as the optimizer and Sparse Categorical Cross Entropy is used for loss.

In experiments 1-5, LSTMs are used and the vocabulary sizes are different. They have the same architecture with 64 embedding dimensions and bidirectional shape. Experiments 1-3 have the same maximum token length with varying numbers of unique tokens 50, 1,000, and 50,000. Experiment 4 has the same architecture as Experiment 2 but it has additional custom stopwords selected from the top 50 vocabulary from the entire corpus. Experiment 5 has a similar architecture as Experiment 2 but it has a maximum token length of 200.

The rest of the Experiments all have a maximum token length of 96 from the top 1,000 unique tokens. Experiments 6-8 are RNN models with different architectures. Experiment 6 has 256 embedding size, 128 bidirectional shape and a 0.5 dropout layer. Experiment 7 has a similar architecture as Experiment 6 with a 128 node dense layer and 0.3 dropout layer before the

bidirectional layer. Experiment 8 has a similar architecture as Experiment 6 but with 2 bidirectional layers, where the additional layer has a shape of 256 nodes.

Experiments 9-11 are LSTM models with different architectures and they have similar architectures as Experiments 6-8 respectively.

Experiments 12 and 13 are CNN models with 256 embedding sizes and a dense layer with 256 nodes. Experiment 12 has 1 1D convolutional layer of filter size 32, 0.5 dropout and maxpooling size 2. Experiment 13 is similar with an additional convolutional layer of filter size 64, dropout and maxpooling layer.

The articles in this dataset are tokenized using NLTK and stopwords are downloaded from NLLK's English stopwords. The accuracy and loss for training, validation, and testing, confusion matrix are calculated and visualized with matplotlib.

Results

Experiment	NN type (**R**NN/**L**STM 1D /**C**NN)	Vocabulary size (input, output)	Architecture (embedding dimension, direction dimension/Conv1D filter)	Regularization	Run time	Training accuracy	Validation accuracy	Testing accuracy	Training loss	Validation loss	Testing loss
1	L	50, 96	64, 64	/	7m	.605	.597	.603	.916	.934	.942
2	L	1000, 96	64, 64	/	7m	.888	.868	.865	.314	.378	.380
3	L	50000, 96	64, 64	/	4m	.954	.908	.910	.145	.301	.288
4	L	1000, 96	64, 64	/	7m	.889	.865	.862	.310	.377	.378
5	L	1000, 200	64, 64	/	7m	.889	.864	.865	.312	.389	.394
6	R	1000, 96	256, 128	0.5 Dropout after Bidirectional layer	8m	.872	.850	.842	.372	.437	.454
7	R	1000, 96	256, 128, Dense layer (128) before Bidirectional layer	0.3 Dropout after Dense layer 0.5 Dropout after Bidirectional layer	7m	.869	.849	.843	.379	.443	.437
8	R	1000, 96	256, 2 Bidirectional layers [128, 256]	0.5 Dropout after Bidirectional layer	20m	.861	.855	.856	.397	.417	.425
9	L	1000, 96	256, 128	0.5 Dropout after Bidirectional layer	6m	.891	.865	.866	.309	.390	.396
10	L	1000, 96	256, 128, Dense layer (128) before Bidirectional layer	0.3 Dropout after Dense layer 0.5 Dropout after Bidirectional layer	5m	.884	.865	.866	.331	.383	.390
11	L	1000, 96	256, 2 Bidirectional layers [128, 256]	0.5 Dropout after Bidirectional layer	11m	.903	.859	.863	.280	.448	.456
12	C	1000, 96	256, 32, Dense (256) layer	0.5 Dropout before Maxpooling layer Pool size = 2	3m	.848	.854	.850	.432	.441	.448
13	C	1000, 96	256, [32, 64] Dense (256) layer	[0.5 Dropout before Maxpooling layer Pool size = 2] after each Conv1D layer	5m	.858	.860	.850	.407	.425	.438

Experiment 1 - 50 most frequent tokens, maximum of 96 tokens. Low accuracy around 60%, high loss around .93, and high RMSE 1.18. Model is underfitting from training and validation curves and classification is okay, with misclassification between all classes.

Experiment 2 - Similar model architecture as Experiment 1 but 1,000 most frequent tokens. Accuracy around 87% , loss around .36, RMSE .64. The performance metrics are much better than Experiment 1 and classification is also much better, with some misclassification between Business and Sci/Tech.

Experiment 3 - Similar model architecture as Experiment 1 but 50,000 most frequent tokens. Highest accuracy of all models around 91%, loss around .3, RMSE .53. The performance metrics are the best so far. Similar classification performance as Experiment 2 but the model is overfitting.

Experiment 4 - Same model as Experiment 2 with custom stopwords selected from the top 50 vocabulary from the entire corpus (in addition to the ‘english’ stopwords from NLTK. Top 20 terms were identified using TF-IDF and the terms are already included in either the english stopwords or the custom stopwords. Accuracy is about 87%, loss around .35, and RMSE .65. Performance, model fit, and classification is comparable to Experiment 2 without much difference. Runtime is the same as well.

Experiment 5 - Similar model as Experiment 2 with maximum of 200 tokens. Like Experiment 4, Performance, classification, runtime are similar and comparable to Experiment 2.

Experiment 6 - This model is RNN, with 256 embedding size and 128 bidirectional shape and a dropout layer. Accuracy around 84%, loss around .43, RMSE .72. The performance of this model is not as good as the previous models and runtime is longer at 8min. Model fit is good and classification is like the others, with some misclassification between Business and Sci/Tech.

Experiment 7 - Similar architecture as Experiment 6 with an extra dense and dropout layer. The performance is worse than Experiment 6 with accuracy around 84%, higher loss and RMSE.

Experiment 8 - Similar architecture as Experiment 6 with an extra bidirectional layer of shape 256. This model took the longest to run out of all models for 20min. The accuracy is 86%, loss is around .4, RMSE is .66. The performance is slightly better but the runtime is 3 times that of Experiment 6 and the model is underfitting, which is not worth the trade-off.

Experiment 9 - Same model as Experiment 6 but it's LSTM instead of RNN. This model is slightly faster than most of the models run so far. The accuracy is 87%, loss is around .35, and RMSE .65. Performs better than Experiment 6, with good model fit and classification.

Experiment 10 - Same model as Experiment 7 but on LSTM model. The Accuracy is 87%, loss around .34, and RMSE .62. Performs slightly better than Experiment 7 but worse than Experiment 9 and also runs faster. Model has a good fit and classification.

Experiment 11 - Same model as Experiment 8 but on LSTM model. Accuracy 86%, loss around .35, and RMSE .66. Similar performance as Experiment 8 and runs twice as fast, but still a slower model compared to the rest. Model is overfitting but has good classification.

Experiment 12 - This is a 1D Convolutional Neural Network with a filter size of 32, 0.5 dropout and pool size 2. This model ran the fastest with an accuracy of 85%. The loss is around .4 and RMSE .67. This model is not as good as the other models (except Experiment 1) but given the runtime and classification, the performance is pretty good. However the model is slightly underfitting.

Experiment 13 - This has similar architecture as Experiment 12 but with 2 convolutional, dropout, and maxpooling layers. The performance is similar to Experiment 12 with a smaller loss. It runs a bit longer for 5min and the model underfits. The classification is good.

Conclusions

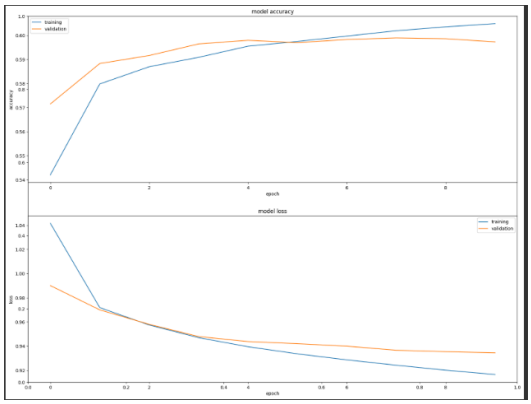
The model from Experiment 10 performed the best overall out of all 13 experiments. Although it does not have the highest testing accuracy and lowest testing loss, it can classify the articles to their corresponding category well. The model also shows an appropriate fitting and the runtime is relatively fast. The model from Experiment 2 is simpler and performed almost as well as Experiment 10 but it runs slower with a higher error. Experiment 10 has an additional dense and dropout layer and higher embedding size compared to Experiment 2.

Most models ran for around 4-7 minutes and have good accuracy and loss curves. Among the better performing models, there is trouble classifying between Business and Sci/Tech articles. Overall, LSTMs performed better than RNNs, which performed better than 1D CNNs. The average accuracy of all models is about 86%, loss around 0.3 and RMSE around 0.6. Some models achieved >90% for training accuracy but they are often overfitting. For future experiments, higher embedding size, additional hidden layers with dropout regularization can be explored for LSTMs to increase the accuracy of the models. Epoch size and batch number can be increased to understand their effects on the models.

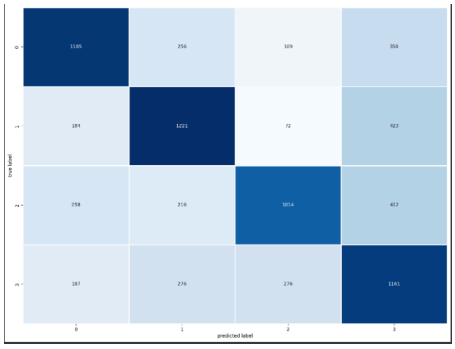
This research demonstrates Long-Short Term memory networks (LSTM) are much better at NLP text classification tasks than Recurrent Neural Networks (RNNs) or Convolutional Neural Networks (CNNs). LSTMs' ability to retain data for later proves its strength over RNNs through better model performance metrics and text classification while keeping the runtime short and

Appendix

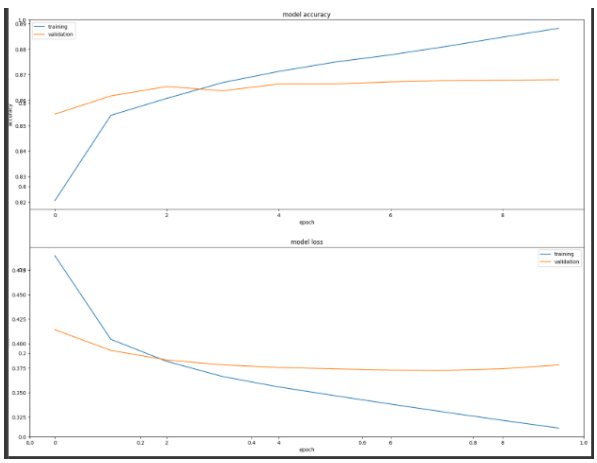
1. Experiment 1: Training & Validation Accuracy/Loss, Classification Report, Confusion Matrix



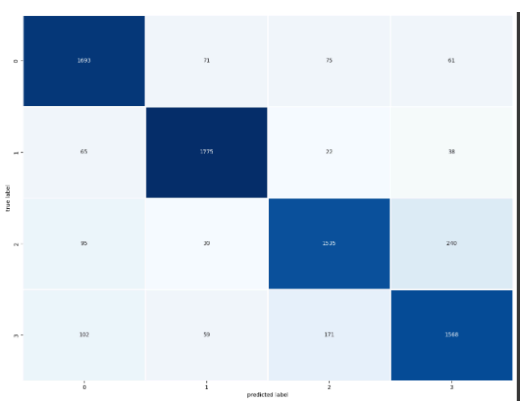

```
Classification Report
              precision    recall  f1-score   support

           0       0.65      0.62      0.64      1900
           1       0.62      0.64      0.63      1900
           2       0.69      0.53      0.60      1900
           3       0.49      0.61      0.55      1900

    accuracy                           0.60      7600
   macro avg       0.61      0.60      0.60      7600
weighted avg       0.61      0.60      0.60      7600

Accuracy Score: 0.6027631578947369
Root Mean Square Error: 1.1761332720760052
```



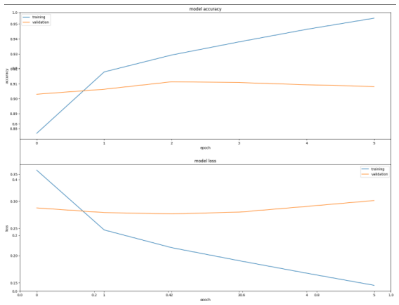
2. Experiment 2: Training & Validation Accuracy/Loss, Classification Report, Confusion Matrix



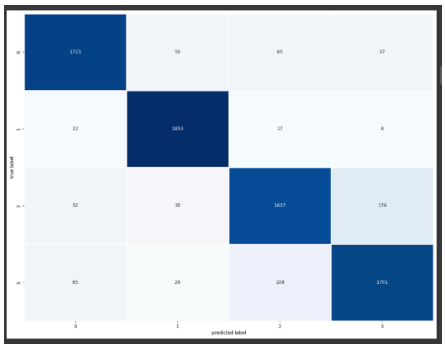

```
Classification Report
              precision    recall  f1-score   support

           0       0.87      0.89      0.88      1900
           1       0.92      0.93      0.93      1900
           2       0.85      0.81      0.83      1900
           3       0.82      0.83      0.82      1900

    accuracy                           0.86      7600
   macro avg       0.86      0.86      0.86      7600
weighted avg       0.86      0.86      0.86      7600

Accuracy Score: 0.8646052631578948
Root Mean Square Error: 0.642159186691767
```



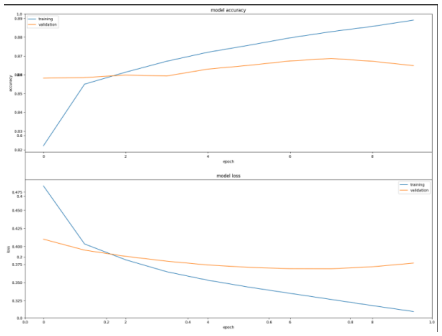
3. Experiment 3: Training & Validation Accuracy/Loss, Classification Report, Confusion Matrix



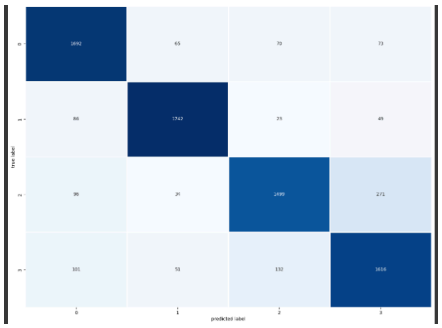

```
Classification Report
              precision    recall  f1-score   support

           0       0.93      0.91      0.92      1900
           1       0.94      0.98      0.96      1900
           2       0.90      0.86      0.88      1900
           3       0.88      0.90      0.89      1900

    accuracy                           0.91      7600
   macro avg       0.91      0.91      0.91      7600
weighted avg       0.91      0.91      0.91      7600

Accuracy Score: 0.9097368421052632
Root Mean Square Error: 0.5275314907038352
```



4. Experiment 4: Training & Validation Accuracy/Loss, Classification Report, Confusion Matrix, TFIDF, top 50 vocab




```
Classification Report
              precision    recall  f1-score   support

           0       0.86      0.89      0.87      1900
           1       0.92      0.92      0.92      1900
           2       0.87      0.79      0.83      1900
           3       0.80      0.85      0.83      1900

    accuracy                           0.86      7600
   macro avg       0.86      0.86      0.86      7600
weighted avg       0.86      0.86      0.86      7600

Accuracy Score: 0.8617105263157895
Root Mean Square Error: 0.6530293779157662
```

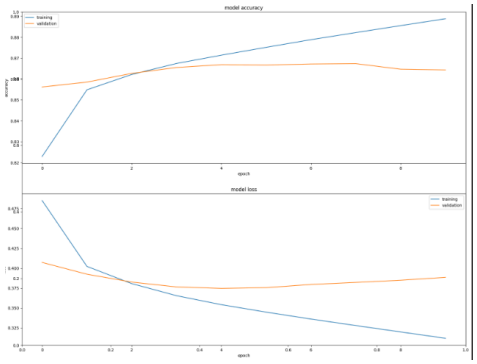


Term	TF-IDF
the	12972.124277
to	8127.768257
of	7930.291075
in	7179.120760
and	6460.573596
on	5226.131594
for	4681.274141
39	4423.080870
that	3752.596143
with	3388.575032

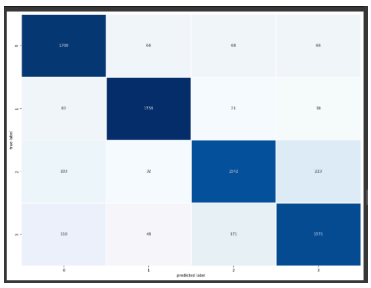
```
['' '[UNK]' '39s' 'said' 'new' 'us' 'reuters' 'ap' 'two' 'first' 'monday'
 'wednesday' 'tuesday' 'thursday' 'company' 'friday' 'inc' 'one' 'world'
 'yesterday' 'last' 'york' 'year' 'president' 'million' 'oil' 'corp'
 'united' 'sunday' 'would' 'week' 'years' 'today' 'three' 'people'
 'government' 'could' 'quot' 'group' 'time' 'game' 'percent' 'saturday'
 'software' 'night' 'iraq' 'prices' 'next' 'security' 'minister']
```

```
CUSTOM_STOPWORDS = [
    '', '[UNK]', '39s', 'said', 'new',
    'us', 'reuters', 'ap', 'two', 'first',
    'monday', 'wednesday', 'tuesday', 'thursday', 'company',
    'friday', 'inc', 'one', 'world', 'yesterday',
    'last', 'york', 'year', 'president', 'million'
]
```

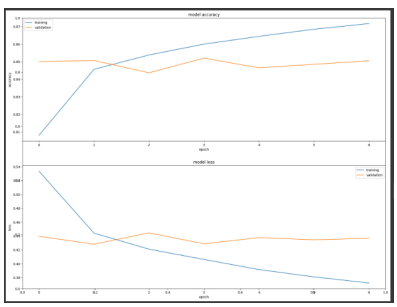
5. Experiment 5: Training & Validation Accuracy/Loss, Classification Report, Confusion Matrix



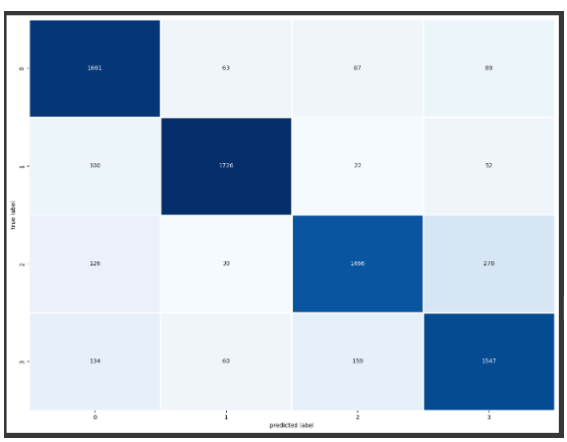

```
Classification Report
              precision    recall  f1-score   support

           0       0.85      0.89      0.87      1900
           1       0.92      0.93      0.92      1900
           2       0.86      0.81      0.83      1900
           3       0.83      0.83      0.83      1900

    accuracy                           0.86      7600
   macro avg       0.86      0.86      0.86      7600
weighted avg       0.86      0.86      0.86      7600

Accuracy Score: 0.8647368421052631
Root Mean Square Error: 0.6495950155370085
```



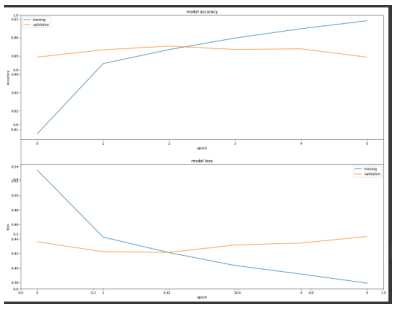
6. Experiment 6: Training & Validation Accuracy/Loss, Classification Report, Confusion Matrix



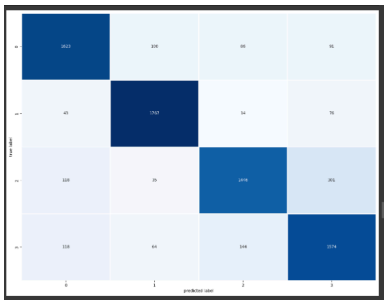

```
Classification Report
              precision    recall  f1-score   support

           0       0.82      0.87      0.85      1900
           1       0.92      0.91      0.91      1900
           2       0.85      0.77      0.81      1900
           3       0.79      0.81      0.80      1900

    accuracy                           0.84      7600
   macro avg       0.84      0.84      0.84      7600
weighted avg       0.84      0.84      0.84      7600

Accuracy Score: 0.8421052631578947
Root Mean Square Error: 0.7217486076409008
```



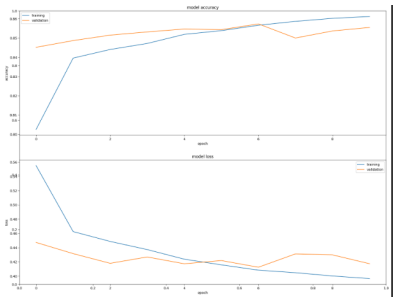
7. Experiment 7: Training & Validation Accuracy/Loss, Classification Report, Confusion Matrix



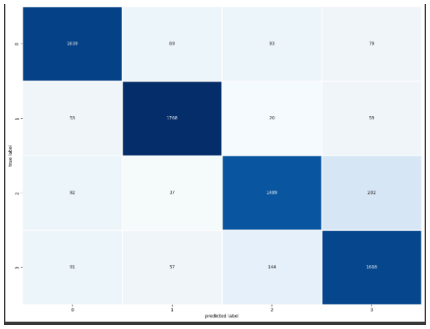

```
Classification Report
              precision    recall  f1-score   support

           0       0.85      0.85      0.85      1900
           1       0.90      0.93      0.91      1900
           2       0.86      0.76      0.81      1900
           3       0.77      0.83      0.80      1900

    accuracy                           0.84      7600
   macro avg       0.84      0.84      0.84      7600
weighted avg       0.84      0.84      0.84      7600

Accuracy Score: 0.843421052631579
Root Mean Square Error: 0.7157991485414268
```



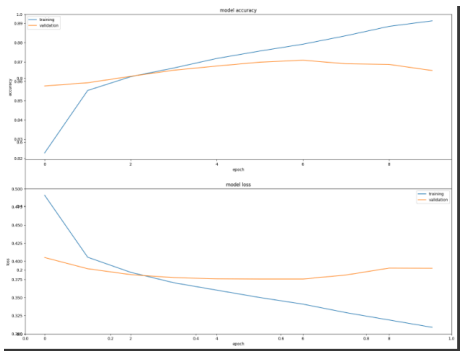
8. Experiment 8: Training & Validation Accuracy/Loss, Classification Report, Confusion Matrix



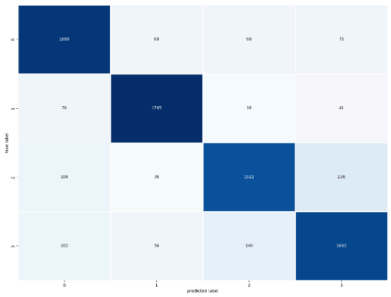

```
Classification Report
              precision    recall  f1-score   support

           0       0.87      0.86      0.87      1900
           1       0.91      0.93      0.92      1900
           2       0.85      0.78      0.82      1900
           3       0.79      0.85      0.82      1900

    accuracy                           0.86      7600
   macro avg       0.86      0.86      0.86      7600
weighted avg       0.86      0.86      0.86      7600

Accuracy Score: 0.8557894736842105
Root Mean Square Error: 0.6648110139058515
```



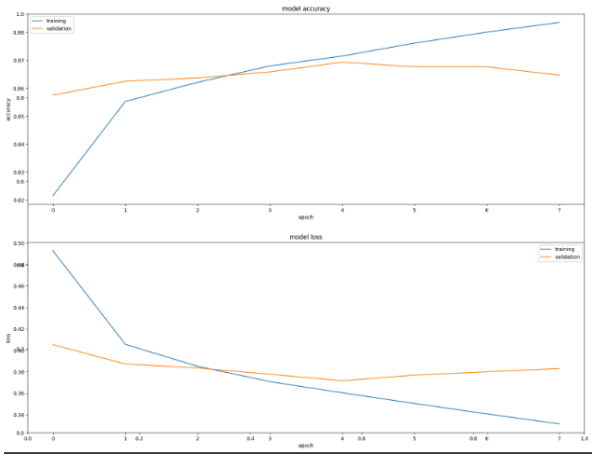
9. Experiment 9: Training & Validation Accuracy/Loss, Classification Report, Confusion Matrix



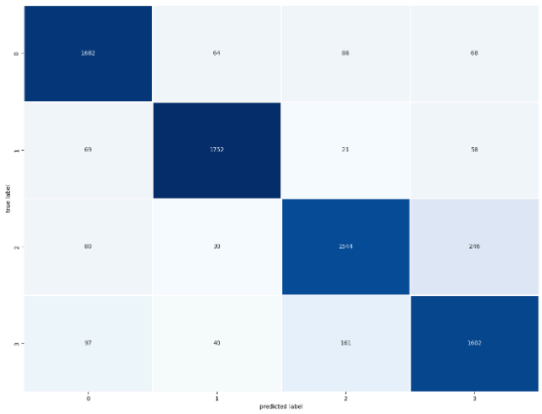

```
Classification Report
              precision    recall  f1-score   support

           0       0.86      0.89      0.87      1900
           1       0.92      0.93      0.92      1900
           2       0.87      0.80      0.83      1900
           3       0.82      0.84      0.83      1900

    accuracy                           0.87      7600
   macro avg       0.87      0.87      0.87      7600
weighted avg       0.87      0.87      0.87      7600

Accuracy Score: 0.8655263157894737
Root Mean Square Error: 0.6540360524426286
```



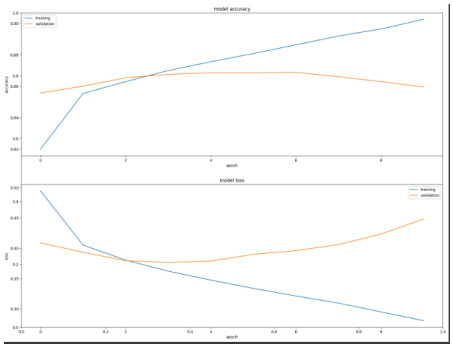
10. Experiment 10: Training & Validation Accuracy/Loss, Classification Report, Confusion Matrix



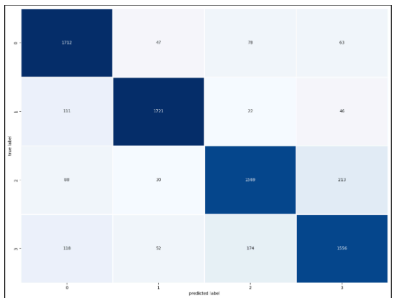

```
Classification Report
              precision    recall  f1-score   support

           0       0.87      0.89      0.88      1900
           1       0.93      0.92      0.93      1900
           2       0.85      0.81      0.83      1900
           3       0.81      0.84      0.83      1900

    accuracy                           0.87      7600
   macro avg       0.87      0.87      0.87      7600
weighted avg       0.87      0.87      0.87      7600

Accuracy Score: 0.8657894736842106
Root Mean Square Error: 0.6419542531659828
```



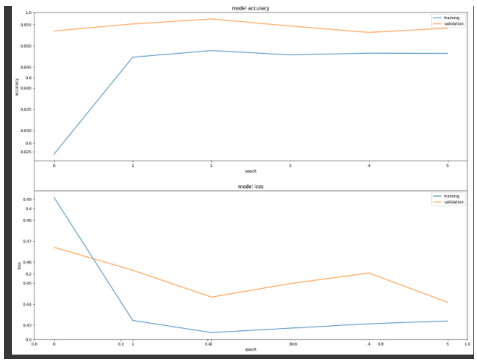
11. Experiment 11: Training & Validation Accuracy/Loss, Classification Report, Confusion Matrix



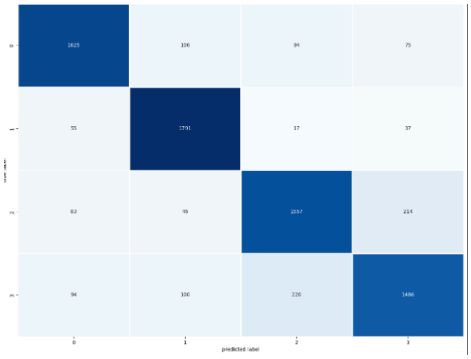

```
Classification Report
              precision    recall  f1-score   support

           0       0.84      0.90      0.87      1900
           1       0.93      0.91      0.92      1900
           2       0.85      0.83      0.84      1900
           3       0.83      0.82      0.82      1900

    accuracy                           0.86      7600
   macro avg       0.86      0.86      0.86      7600
weighted avg       0.86      0.86      0.86      7600

Accuracy Score: 0.8628947368421053
Root Mean Square Error: 0.6571469434328656
```



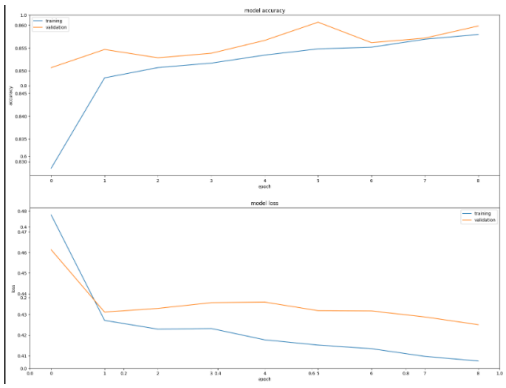
12. Experiment 12: Training & Validation Accuracy/Loss, Classification Report, Confusion Matrix



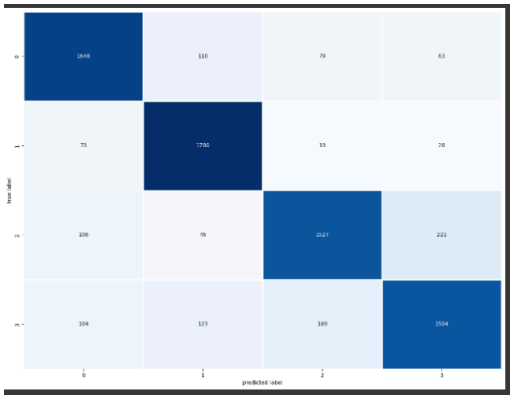

```
Classification Report
              precision    recall  f1-score   support

           0       0.88      0.86      0.87      1900
           1       0.88      0.94      0.91      1900
           2       0.82      0.82      0.82      1900
           3       0.82      0.78      0.80      1900

    accuracy                           0.85      7600
   macro avg       0.85      0.85      0.85      7600
weighted avg       0.85      0.85      0.85      7600

Accuracy Score: 0.8498684210526316
Root Mean Square Error: 0.6722898810859244
```



13. Experiment 13: Training & Validation Accuracy/Loss, Classification Report, Confusion Matrix




```
Classification Report
              precision    recall  f1-score   support

           0       0.85      0.87      0.86      1900
           1       0.86      0.94      0.90      1900
           2       0.85      0.80      0.83      1900
           3       0.83      0.79      0.81      1900

    accuracy                           0.85      7600
   macro avg       0.85      0.85      0.85      7600
weighted avg       0.85      0.85      0.85      7600

Accuracy Score: 0.8498684210526316
Root Mean Square Error: 0.6771651435055908
```



References

Built In. (2021, January 29). Recurrent Neural Networks and LSTM. Built In. Retrieved from https://builtin.com/data-science/recurrent-neural-networks-and-lstm

GeeksforGeeks. (n.d.). Deep learning – Introduction to Long Short-Term Memory. GeeksforGeeks. Retrieved from https://www.geeksforgeeks.org/deep-learning-introduction-to-long-short-term-memory/

Gulli, A., & Ferro, L. (2006). The AG News corpus. http://groups.di.unipi.it/~gulli/AG_corpus_of_news_articles.html

Luan, Y., & Lin, S. (2019, March). Research on text classification based on CNN and LSTM. In 2019 IEEE international conference on artificial intelligence and computer applications (ICAICA) (pp. 352-355). IEEE.

Pang, B., & Lee, L. (2004). A sentimental education: Sentiment analysis using subjectivity summarization based on minimum cuts. arXiv preprint cs/0409058.

Papers with Code. (n.d.). AG News. Papers with Code. Retrieved from https://paperswithcode.com/dataset/ag-news